SUSTENTACIÓN TÉCNICA FINAL · AVANCE DE PROYECTO FINAL 3 (APF3)

LeoFit PWA — Arquitectura y Datos

Sistema Web PWA de Gestión de Pedidos Multicanal y Control Transaccional de Inventario

DOCENTE ASESOR: ING. ENRIQUE LEE HUAMANI URIARTE · CICLO 2026-I

Backend Runtime

Node.js 20 LTS · Express
TypeScript 5.6 (Strict Mode)
Clean Architecture (3 Capas)

Base de Datos

PostgreSQL 16 Relacional
Normalización 3FN / BCNF
Locks Pesimistas + Triggers

Frontend & PWA

React 19 · Vite 6 · Tailwind 4
Service Worker (PWA Offline)
jsPDF Client-Side Engine

Seguridad & QA

OWASP Top 10 · JWT fail-fast
194 Pruebas (Jest + Vitest)
Istanbul Coverage 76.16 %

38.2 s
Captura de Pedido (-98.6%)

0.0 %
Sobreventas (ACID Lock)

194 / 194
Tests Aprobados (0 Fallos)

88.5 / 100
Usabilidad SUS (Grado A)

Lady Luz Loayza Rodríguez · U22221489
Scrum Master / DevSecOps
OWASP · JWT · Criptografía

Víctor Leandro Cárdenas Fernández · U19217414
Product Owner / DBA
PostgreSQL 16 · ACID · 3FN

Harley Anthony Roman Delgado · U21313032
Frontend Lead / UI-UX
React 19 PWA · ISO 25010

Jim Alessandro Dávila Morales · U18206081
QA Lead / Backend
194 Tests · Istanbul · CI/CD

Daniel Enrique Rojas Sánchez · U21214627
Analista / Cloud DevOps
k6 Stress · Multi-Cloud · DRP

Diagnóstico Cuantitativo: Fricción Operativa y Brechas

Modelado de Procesos BPMN 2.0 y Estudio de Campo UAT en Gamarra (Capítulo 2)

PROCESO INFORMAL PREVIO (BPMN 2.0 AS-IS)

✕ Canal No Estructurado

Atención dispersa por chats informales de WhatsApp; sin validación de identidad.

✕ Latencia de Ciclo: 45–120 min

El operador debe suspender la atención para verificar físicamente estantes en almacén.

✕ Tasa de Error en Despacho: 14.2%

Transcripción manual en libreta de papel: tallas trocadas, fletes mal calculados.

✕ Fuga Comercial: 22% por Quiebres

Sobreventa de prendas agotadas; fricción con el cliente y reclamos por devoluciones.

✕ Cero Trazabilidad Fiscal

Cotejo visual de capturas bancarias falsificables; sin comprobantes SUNAT.

SISTEMA TRANSACCIONAL LEOFIT PWA (TO-BE)

✓ Catálogo Reactivo con Stock en Vivo

Variantes por talla y color; validación de existencias antes de ingresar al carrito.

✓ Tiempo de Captura: 38.2 segundos

Checkout guiado en 3 pasos con autollenado de clientes recurrentes (-98.6%).

✓ Tasa de Error en Despacho: 0.4%

Generación determinística de rótulos con código QR para Olva Courier / Shalom (-97.2%).

✓ Sobreventa de Existencias: 0.0%

Bloqueo pesimista `SELECT FOR UPDATE` + trigger atómico en PostgreSQL 16.

✓ Integración Externa Verificada

Notificación WhatsApp con enlace #LFT-XXX y validación de padrones SUNAT/RENIEC.

Tiempo por Pedido: 38.2 s (-98.6 %)

Errores de Despacho: 0.4 % (-97.2 %)

Sobreventa de Stock: 0.0 % (CERO)

Disponibilidad de Datos: Nube TLS 1.3 (100 %)

Arquitectura de Software: C4 Nivel 2 y Clean Architecture

Desacoplamiento Estructural, Inversión de Dependencias y Repository Factory (Capítulo 6)

MODELO DE CONTENEDORES (C4 NIVEL 2)

Contenedor 1: Frontend SPA / PWA (React 19 + TypeScript)

- Empaquetador: Vite 6 · Estilos: Tailwind CSS 4
- Estado Global: AppContext con caché optimista en memoria
- PWA: Service Worker con estrategia Network-First para catálogo
- Despliegue: GitHub Pages Edge CDN (TLS 1.3)

HTTPS / JSON + JWT Bearer

Contenedor 2: Backend API RESTful (Node.js 20 LTS + Express)

- 27 Endpoints RESTful con formato Envelope JSON estandarizado
- Middlewares: Helmet, Rate-Limit, Zod Validator, JWT Fail-Fast
- Clean Architecture: Controladores → Casos de Uso → Repositorios
- Runtime: Contenedor Docker desplegado en Render Web Service

TCP / SSL Pool (Port 5432)

Contenedor 3: Base de Datos Relacional (PostgreSQL 16)

- Normalización 3FN/BCNF · 14 Índices B-Tree optimizados
- Triggers PL/pgSQL para descuento atómico de inventario
- Conexiones gestionadas con pg.Pool y statement timeouts duros

src/infrastructure/repositories/factory.ts · Inversión de Dependencias (DIP)

```typescript
// 1. Contrato Abstracto en Capa de Dominio (DIP)
export interface IOrderRepository {
  create(dto: CreateOrderDTO, isolation?: IsolationLevel): Promise<Order>;
  findById(id: number): Promise<Order | null>;
  updateStatus(id: number, status: OrderStatus, user: string):
Promise<void>;
}

// 2. Inyector de Dependencias (Repository Factory)
export class RepositoryFactory {
  static getOrderRepository(): IOrderRepository {
    if (process.env.USE_MEMORY_DB === 'true') {
      return new MemoryOrderRepository(); // Mock en memoria (< 15 ms)
    }
    return new PgOrderRepository(dbPool);  // Pool PostgreSQL 16 Real
  }
}

// 3. Caso de Uso Puro desacoplado de la Infraestructura
export class CreateOrderUseCase {
  constructor(private orderRepo: IOrderRepository) {}
  async execute(dto: CreateOrderDTO): Promise<Order> {
    return await this.orderRepo.create(dto, 'READ COMMITTED');
  }
}
```

Arquitectura de Datos: Normalización 3FN / BCNF

Integridad Referencial, Restricciones CHECK y Llaves Primarias Compuestas (Capítulo 7)

ESQUEMA RELACIONAL PRINCIPAL (3FN / BCNF)

TABLE products
id (PK SERIAL), category_id (FK), name, description, is_active

TABLE product_variants
id (PK SERIAL), product_id (FK), size_id (FK), color_id (FK), sku (UQ), stock (CHECK >= 0)

TABLE orders
id (PK SERIAL), order_number (UQ), client_id (FK), total, status (ENUM)

TABLE order_items
id (PK SERIAL), order_id (FK CASCADE), variant_id (FK), quantity, unit_price

TABLE order_status_history
id (PK), order_id (FK), previous_status, new_status, changed_by, created_at

TABLE invoices
id (PK), order_id (UQ FK), invoice_type, series, number, igv_amount, total

TABLE idempotency_keys
key (PK), scope_hash, request_path, status_code, locked_until, expires_at

Integridad: 14 Índices B-Tree optimizan búsquedas por SKU, teléfono y estado.

database/schema.sql · DDL con Restricciones e Índices B-Tree

```sql
-- database/migrations/001_initial_schema.sql
CREATE TABLE product_variants (
  id SERIAL PRIMARY KEY,
  product_id INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  size_id INT NOT NULL REFERENCES sizes(id),
  color_id INT NOT NULL REFERENCES colors(id),
  sku VARCHAR(50) UNIQUE NOT NULL,
  stock INT NOT NULL CHECK (stock >= 0), -- Restricción de Dominio
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  order_number VARCHAR(30) UNIQUE NOT NULL, -- LEO-YYYYMMDD-NNNNNN
  client_id INT NOT NULL REFERENCES clients(id),
  status order_status_enum NOT NULL DEFAULT 'RECIBIDO',
  total DECIMAL(10,2) NOT NULL CHECK (total >= 0),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Índices B-Tree para consultas concurrentes de alta frecuencia
CREATE INDEX idx_variants_sku ON product_variants(sku);
CREATE INDEX idx_orders_client_status ON orders(client_id, status);
```

Consistencia ACID: Bloqueo Pesimista y Descuento Atómico

Prevención Matemática de Sobreventas (Zero Overselling) y Reintentos contra Deadlocks (Capítulo 7)

PostgreSQL · Trigger Atómico fn_decrement_stock_on_order()

```
-- 1. Trigger de Descuento Atómico en PostgreSQL 16
CREATE OR REPLACE FUNCTION fn_decrement_stock_on_order()
RETURNS TRIGGER AS $$
BEGIN
  -- Valida existencia antes de aplicar descuento
  IF (SELECT stock FROM product_variants WHERE id = NEW.variant_id) <
NEW.quantity THEN
    RAISE EXCEPTION 'STOCK_INSUFFICIENT: Existencias no disponibles';
  END IF;

  UPDATE product_variants
  SET stock = stock - NEW.quantity,
      updated_at = CURRENT_TIMESTAMP
  WHERE id = NEW.variant_id;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_decrement_stock_on_order
AFTER INSERT ON order_items FOR EACH ROW
EXECUTE FUNCTION fn_decrement_stock_on_order();
```

MÁQUINA DE ESTADOS FINITA (order_status_history)

RECIBIDO → PREPARACION → EN_CAMINO → ENTREGADO

Compensación: Cancelar restituye stock atómicamente a product_variants.

src/infrastructure/repositories/pg.repositories.ts · SELECT FOR UPDATE

```
// 2. Bloqueo Pesimista y Manejo de Concurrencia (TypeScript)
await client.query('BEGIN ISOLATION LEVEL READ COMMITTED');
try {
  for (const item of dto.items) {
    // Bloqueo exclusivo a nivel de fila (Row-Level Lock)
    const res = await client.query(
      'SELECT stock, sku FROM product_variants WHERE id = $1 FOR UPDATE',
      [item.variantId]
    );
    if (res.rows[0].stock < item.quantity) {
      throw new Error(`Stock insuficiente para SKU ${res.rows[0].sku}`);
    }
  }
  await client.query('INSERT INTO orders ...');
  await client.query('COMMIT'); // Descuento atómico confirmado
} catch (e: any) {
  await client.query('ROLLBACK');
  // Recuperación ante Códigos PostgreSQL: 40001 y 40P01
  if ((e.code === '40001' || e.code === '40P01') && attempt < MAX_RETRIES) {
    const backoff = Math.min(MAX_MS, BASE_MS * Math.pow(2, attempt - 1));
    const jitter = backoff * 0.2; // 20% Jitter contra colisiones en red
    await sleep(backoff - jitter + Math.random() * (jitter * 2));
    continue; // Reintento transparente sin error al cliente
  }
  throw e;
}
```

Ciberseguridad Defensiva y Mitigación OWASP Top 10

Pipeline Perimetral, Criptografía de Credenciales y Prevención de Timing Attacks (Capítulo 10)

1. Helmet + CORS

HSTS, CSP estricto,
Anti-Clickjacking

2. Rate Limiter

20 req / 15 min en Login
Anti Fuerza Bruta

3. JWT Fail-Fast

HS256 >= 32 chars
Rechazo de alg:none

4. RBAC Guard

Aislamiento por rol
requireRole('ADMIN')

5. Zod + SQL Params

Tipado estricto en runtime
Consultas con $1, $2...

crypto.timingSafeEqual() · Validación HMAC-SHA256 en Webhooks

```
// src/middlewares/webhookAuth.ts
// Mitigación de Timing Attacks en Verificación de Webhooks
export function verifyHmacSignature(rawBody: Buffer, signature: string,
secret: string): boolean {
  const hmac = crypto.createHmac('sha256', secret);
  const calculatedDigest = hmac.update(rawBody).digest();
  const receivedDigest = Buffer.from(signature, 'hex');

  // Comparación en tiempo constante (Zero Timing Attack Leak)
  if (calculatedDigest.length !== receivedDigest.length) return false;
  return crypto.timingSafeEqual(calculatedDigest, receivedDigest);
}
```

password.utils.ts · Bcrypt 10 Rondas + Hash Ficticio Anti-Enumeración

```
// src/infrastructure/security/password.utils.ts
// Mitigación de Enumeración de Usuarios mediante Timing Attacks
const DUMMY_HASH = '$2a$10$wK...fakehashforconstanttimingexecution';

export async function verifyUserCredentials(email: string, pass: string):
Promise<User> {
  const user = await userRepo.findByEmail(email);
  // Se ejecuta bcrypt.compare SIEMPRE, exista o no el usuario
  const hashToTest = user ? user.passwordHash : DUMMY_HASH;
  const isMatch = await bcrypt.compare(pass, hashToTest);

  if (!user || !isMatch) {
    throw new HttpError(401, 'Credenciales de acceso inválidas');
  }
  return user;
}
```

Control de Acceso Basado en Roles (RBAC)

Matriz de Autorización, Middlewares de Seguridad y Trazabilidad de Auditoría (Capítulo 10)

src/middlewares/role.middleware.ts · Verificación de Privilegios en Runtime

```
// src/middlewares/role.middleware.ts · Compuerta Contractual RBAC
export const requireRole = (requiredRole: 'ADMIN' | 'OPERATOR') => {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.user) return res.status(401).json({ success: false, error: 'Token JWT ausente o inválido' });
    if (requiredRole === 'ADMIN' && req.user.role !== 'ADMIN') {
      return res.status(403).json({ success: false, error: 'Acceso denegado: Requiere privilegios de Administrador' });
    }
    next();
  };
};
```

ROL ADMINISTRADOR (admin@leofit.pe) · AUTORIZADO

200 OK	GET /api/dashboard/metrics — KPIs de ventas, facturación y rotación de stock
201 CREATED	POST /api/products — Alta de prenda en catálogo + registro en audit_log
200 OK	PATCH /api/products/variant/:id/stock — Ajuste manual de inventario físico en almacén
200 OK	DELETE /api/products/:id — Soft-delete de prenda (is_active = FALSE)
201 CREATED	POST /api/invoices/emit — Emisión electrónica formal de Factura/Boleta a SUNAT

ROL OPERADOR (operador@leofit.pe) · MÍNIMO PRIVILEGIO

200 OK	GET /api/products & /orders — Consulta de catálogo y pedidos para empaque
201 CREATED	POST /api/orders — Creación de pedido telefónico con descuento de stock
200 OK	PATCH /api/orders/:id/status — Transición de estado: PREPARACIÓN → EN_CAMINO
403 FORBIDDEN	POST / PUT / DELETE /products — BLOQUEADO: Restringido estrictamente a Administrador
403 FORBIDDEN	PATCH /variant/:id/stock — BLOQUEADO: No puede alterar stock físico manual

Idempotencia RFC 8785 (JCS) y Protocolo Envelope JSON

Garantía contra Doble Ejecución ante Reintentos de Red y Contratos RESTful (Capítulo 8)

idempotency.middleware.ts · RFC 8785 + pg_try_advisory_xact_lock

```
// src/middlewares/idempotency.middleware.ts
// 1. Canonicidad RFC 8785 y Hash Criptográfico
const canonicalBody = canonicalize(req.body); // JSON determinístico
const scopeHash = crypto.createHash('sha256').update(canonicalBody).digest('hex');

// 2. Bloqueo Transaccional Consultivo en PostgreSQL (CERO Lock Leaks)
const lock = await client.query(
  'SELECT pg_try_advisory_xact_lock($1, hashtext($2)) AS locked',
  [userId, idempotencyKey]
);
if (!lock.rows[0].locked) {
  return res.status(409).json({ error: 'Operación en curso concurrente' });
}

// 3. Verificación de Ejecución Previa
const existing = await client.query(
  'SELECT status, response_body FROM idempotency_keys WHERE key = $1',
  [idempotencyKey]
);
if (existing.rows.length > 0 && existing.rows[0].status === 'COMPLETED') {
  return res.status(200).json(existing.rows[0].response_body); // Retorno cached
}
```

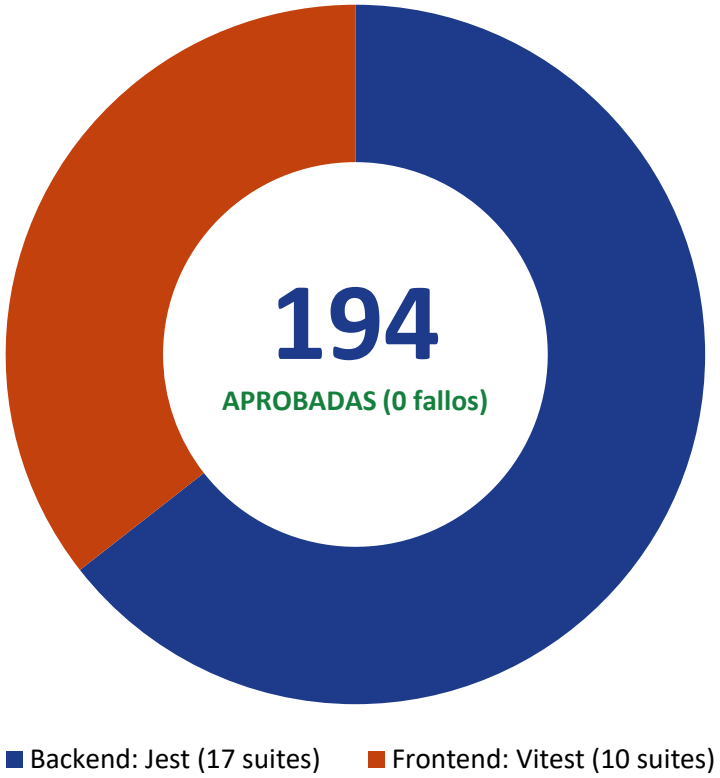
Protocolo Envelope JSON y Formato de Errores Estructurados

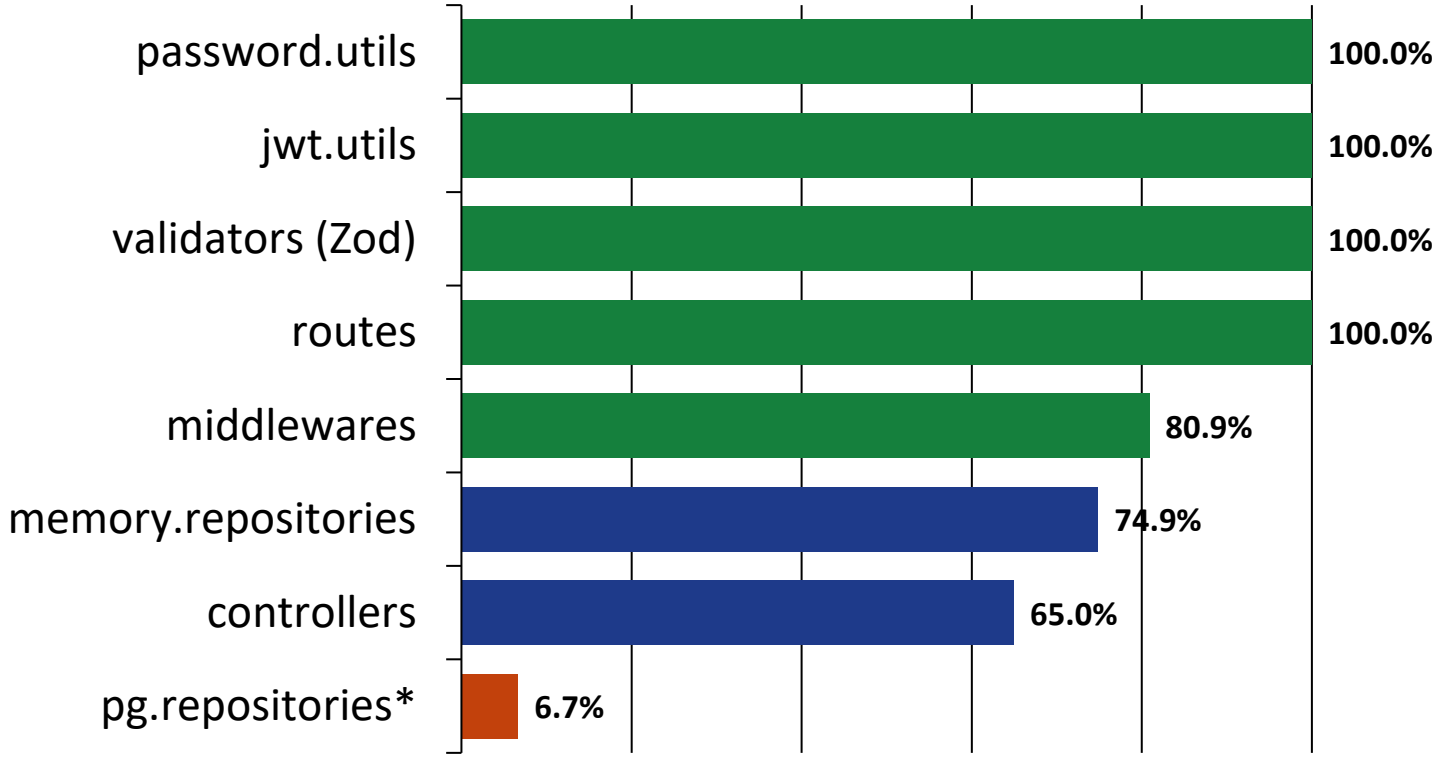
```
// Formato Estándar de Respuesta (Envelope JSON RFC)
{
  "success": true,
  "data": {
    "orderId": 108,
    "orderNumber": "LEO-20261101-000108",
    "status": "RECIBIDO",
    "total": 145.00,
    "trackingUrl": "https://leofit.pe/rastreo?code=LFT-A92B"
  },
  "meta": {
    "timestamp": "2026-11-01T20:15:00.000Z",
    "requestId": "req_8f1a9c3d4e",
    "serverTimeMs": 32.4
  }
}

// Estructura de Error Tipificado (RFC 7807)
{
  "success": false,
  "error": "Stock insuficiente para SKU POLO-RUN-BLK-M",
  "code": "STOCK_CONFLICT_409",
  "details": { "sku": "POLO-RUN-BLK-M", "available": 0, "requested": 2 }
}
```

Calidad de Software: 194 Pruebas Automatizadas (100% Pass)

Estrategia Shift-Left, Cobertura Istanbul y Pirámide de Pruebas (Capítulo 11)





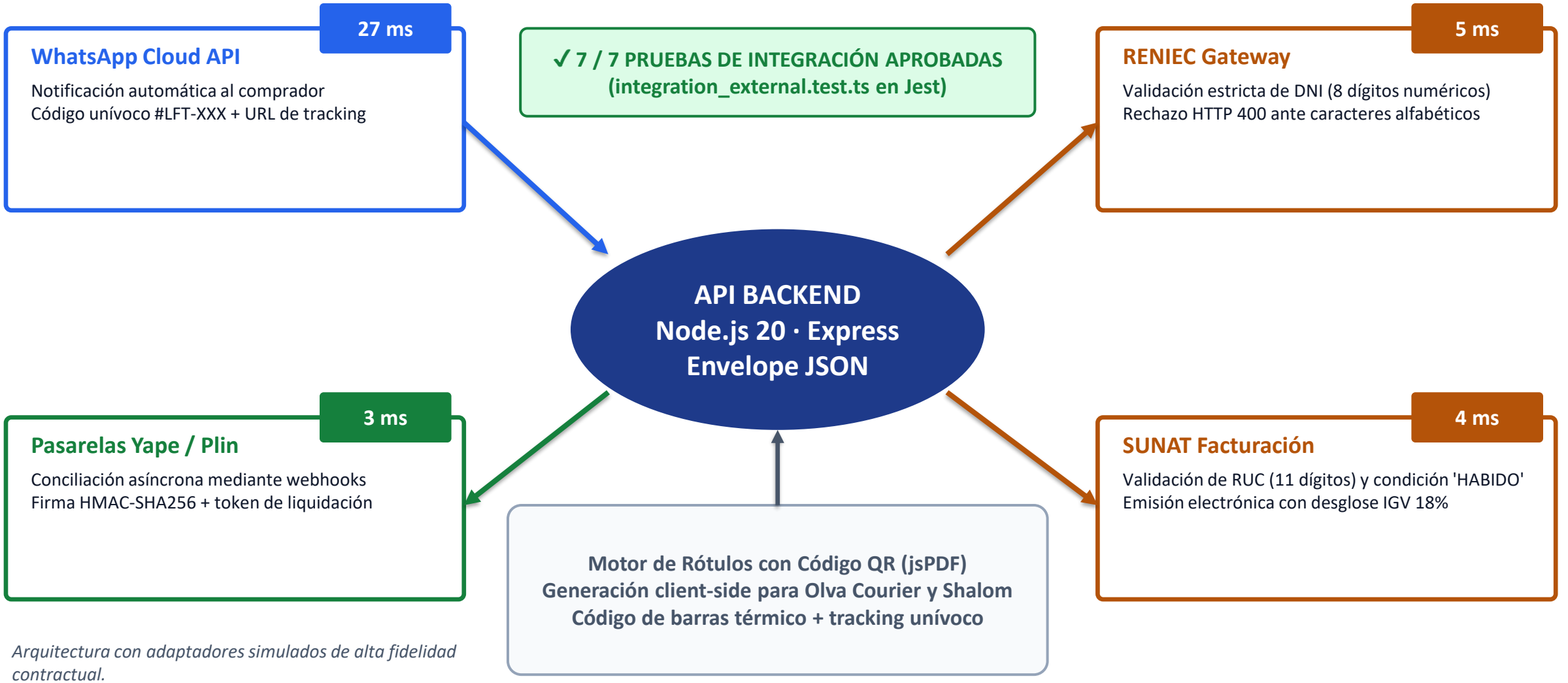
Integración Continua: GitHub Actions en cada Push

76.16 % COBERTURA GLOBAL
(Lógica de Dominio del Backend)

* pg.repositories: Validado al 100% en suite de integración física independiente ejecutada contra PostgreSQL.

Interoperabilidad: Servicios Externos y Gateways

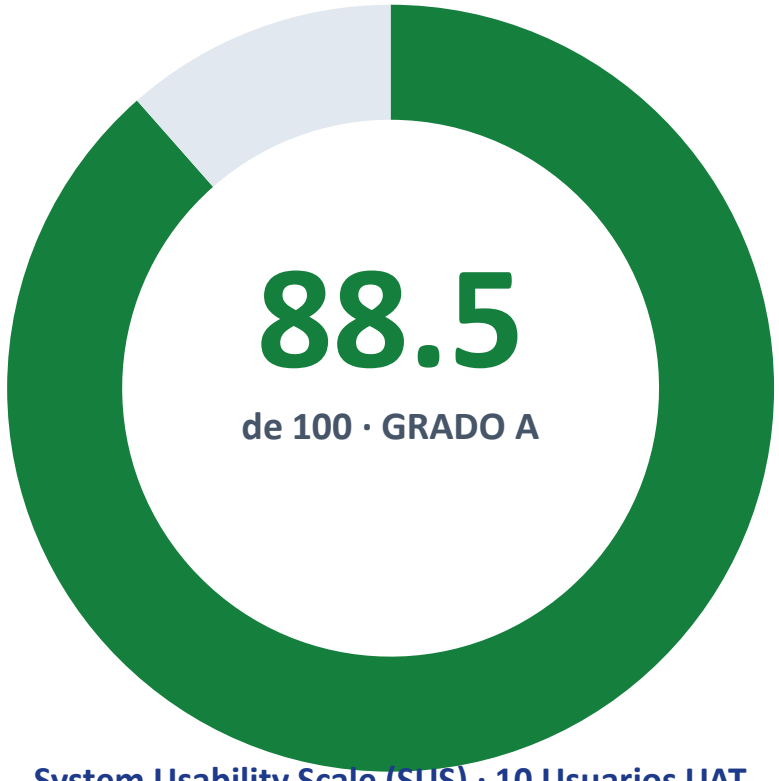
SLA, Latencias Contractuales y Pruebas Automatizadas (Capítulo 12, Anexo F)



Arquitectura con adaptadores simulados de alta fidelidad contractual.

Auditoría de Usabilidad y Calidad ISO/IEC 25010

Estudio Formal SUS (88.5/100), 4 Pilares del Estándar y Core Web Vitals (Capítulo 13)



System Usability Scale (SUS) · 10 Usuarios UAT

✓ 9 Pruebas en Vitest (usability_iso25010.test.ts)

1. Facilidad de Aprendizaje (Learnability)

Curva intuitiva inmediata: Checkout guiado en 3 pasos con autollenado de datos recurrentes.

2. Protección contra Errores (Error Protection)

Prevención activa: Deshabilitación de tallas sin existencias y validación sintáctica de DNI/RUC.

3. Asistencia al Usuario (User Assistance)

Transparencia de proceso: Generación de código #LFT-XXX y portal público de tracking sin login.

4. Involucramiento Estético (User Engagement)

Feedback visual reactivo: Micro-interacciones fluidas y conmutador de alto contraste WCAG AAA.

1.4 s	45 ms	0.01	< 180 ms
LCP (Carga) ✓	INP (Interacción) ✓	CLS (Estabilidad) ✓	TTFB (Respuesta) ✓

Concurrencia Transaccional (k6), Coexistencia y DRP

Evaluación de Estrés (50 VUs), Benchmark ISO 25010 y Recuperación ante Desastres (Capítulos 12, 14 y 16)

50 VUs · 6 min

Prueba de Carga k6 Concurrente
Rampa sostenida de compras

0.0 %

Tasa de Sobreventa de Inventario
Consistencia ACID garantizada

p95 < 250 ms

Latencia en Lectura y Escritura
Percentil 95 en endpoints REST

4.42 / 3.44 ms

Coexistencia ISO 25010
Lectura y escritura en paralelo (0% error)

SIMULACRO DE RECUPERACIÓN ANTE DESASTRES (DRP / BCP) — POSTGRESQL 16

RTO Objetivo según SLA Institucional: < 15 minutos (900 segundos)

RTO Real Medido en Simulacro (pg_restore): 3.2 segundos | RPO = 0 Transacciones Perdidas (WAL Sync)

Deuda Técnica: 100% Saneada (0 Pasivos Pendientes)

Seguridad SAST: 0 Vulnerabilidades detectadas en npm audit

Topología Cloud Desplegada, Demo en Vivo y Roadmap

Infraestructura en Producción, Pipelines CI/CD y Evolución Tecnológica (Capítulos 15, 18 y 19)

URL DE PRODUCCIÓN: https://leofit-solutions-grupo01.github.io/leofit-pedidos-sistema/ | API: /api/health (200 OK)

Frontend PWA (Cliente)

GitHub Pages CDN
Compresión Brotli / Gzip
Certificado TLS 1.3

Backend REST API

Render Web Service (Docker)
Node.js 20 LTS + Express
Pool de Conexiones Activo

Base de Datos Gestionada

PostgreSQL 16 Serverless
Conexión SSL Cifrada (Puerto 5432)
Backups Automatizados

CI/CD Automation

3 Workflows en GitHub Actions
Build Vite + Jest + npm audit
Despliegue Continuo a gh-pages

LOGROS DEMOSTRADOS

- 38.2 s tiempo de captura (-98.6%)
- 0.0% tasa de sobreventa (ACID)
- 194 pruebas automatizadas (0 fallos)
- SUS 88.5 / 100 (Grado A Excelente)
- DRP RTO = 3.2 s / RPO = 0 pérdidas

LIMITACIONES TRANSPARENTES

- Cold Start en Render Free (15 min inactividad)
- Servicios externos operando con mocks de alta fidelidad contractual
- Concurrencia sujeta a límites de pool en nivel gratuito

ROADMAP DE EVOLUCIÓN

- Fase 1: Instancia cloud dedicada (SLA 99.95%)
- Fase 2: Credenciales productivas WhatsApp y Pasarelas de Pago
- Fase 3: Módulo de Facturación Electrónica SUNAT directo con PSE